

SPORTCONNECT

Dossier d'Architecture Technique

POC — Plateforme Sociale pour Sportifs

Architecture polyglotte — 6 bases de données spécialisées

Déploiement sur Google Cloud Platform (GCP)

Version 2.0 — Mars 2026

Sommaire

1. Analyse approfondie des besoins en stockage

- 1.1 Cartographie des données
- 1.2 Patterns d'accès identifiés
- 1.3 Exigences non fonctionnelles

2. Justification détaillée des choix technologiques

- 2.1 Tableau de synthèse
- 2.2 Détail des justifications par technologie

3. Schémas conceptuels et physiques

- 3.1 Schéma relationnel PostgreSQL — DDL physique complet (7 tables)
- 3.2 Schéma documentaire MongoDB — Collection activités
- 3.3 Schéma de graphe Neo4j
- 3.4 Schéma InfluxDB — Séries temporelles capteurs
- 3.5 Schéma conceptuel — Diagramme Entité-Relation (ERD)
- 3.6 Architecture de stockage des médias
- 3.7 Flux temps réel des capteurs IoT
- 3.8 Cohérence des données inter-bases — Pattern Saga

4. Procédures d'administration complètes

- 4.1 Gestion des rôles — Principe du moindre privilège
- 4.2 Monitoring et alertes
- 4.3 Administration Kafka — Topics et consumers
- 4.4 Conformité RGPD — Données de santé
- 4.5 Droit à l'oubli — Suppression en cascade
- 4.6 Initialisation de l'infrastructure

5. Plan de sauvegarde et reprise d'activité (PRA)

- 5.1 Objectifs de récupération (RPO, RTO, MTTR)
- 5.2 Stratégie de sauvegarde par technologie
- 5.3 Scénarios de panne (PostgreSQL, MongoDB, InfluxDB, Kafka, Redis)
- 5.4 Tests de PRA obligatoires

6. Stratégie de mise à l'échelle

- 6.1 Mécanismes de scaling par technologie
- 6.2 Projections de croissance
- 6.3 Stratégie de cache Redis

7. Connexion des technologies à Google Cloud Platform

- 7.1 Mapping technologies / services GCP
- 7.2 Architecture réseau VPC
- 7.3 Configuration détaillée par service (dont Kafka MSK)
- 7.4 Gestion des accès — IAM et Secret Manager

Ann. A Architecture Decision Records (ADR) — dont ADR-005 : Kafka

Ann. B Glossaire technique

1. Analyse approfondie des besoins en stockage

SportConnect est une plateforme sociale dédiée aux sportifs. Son architecture doit orchestrer six catégories de données aux cycles de vie et contraintes techniques radicalement différents. Une analyse préalable est indispensable pour justifier une architecture polyglotte plutôt qu'une solution monolithique.

1.1 Cartographie des données

Tableau 1.1 — Cartographie des données et contraintes

Catégorie	Entité	Volume estimé	Contraintes clés	Pattern d'accès
Données critiques	Profils, Auth	~1M enreg.	ACID, RGPD, confidentialité	Lecture élevée, écriture faible
Données volumétriques	Activités sportives	~50M/an	Schéma variable selon sport	Lecture/écriture équilibrées
Données de réseau	Relations sociales	~10M liens	Traversées multi-niveaux	Lectures complexes fréquentes
Haute fréquence	Capteurs GPS, FC	~500M pts/jour	Ingestion massive, TTL court	Write-heavy, lecture agrégée
Éphémères	Sessions, likes, cache	Variable	Latence < 1ms	Très haute fréquence
Médias	Photos, vidéos	~10 To/an	Taille volumineuse, CDN global	Lecture intensive, upload ponctuel

1.2 Patterns d'accès identifiés

Lecture intensive

- Fil d'actualité (feed) des abonnements — requête la plus fréquente de la plateforme
- Consultation des profils publics et statistiques personnelles de performance
- Classements et leaderboards hebdomadaires par sport

Écriture intensive

- Ingestion des données capteurs GPS et fréquence cardiaque en temps réel (~500M pts/jour)
- Enregistrement des activités à la fin d'une session sportive
- Interactions sociales (likes, commentaires) — pic lors des publications d'activités

Calculs complexes

- Traversées de graphe pour les suggestions d'amis (amis d'amis) — O(k) requis
- Agrégations temporelles de performance (distance hebdomadaire, VO2max estimé)
- Analyse de fréquence cardiaque sur la durée d'une activité (détection d'anomalies)

1.3 Exigences non fonctionnelles

Tableau 1.2 — Exigences non fonctionnelles

Exigence	Objectif	Mesure	Impact architectural
Disponibilité	99,9% SLA	Uptime mensuel	Réplication multi-nœuds, failover automatique
Latence lecture	< 100ms	P95 requêtes API	Cache Redis, index optimisés

Exigence	Objectif	Mesure	Impact architectural
Latence cache	< 10ms	P99 Redis	Memorystore GCP Standard Tier
Scalabilité	x10 sur 3 ans	Volume données	Sharding horizontal natif par SGBD
Sécurité	Conformité RGD	Audit annuel	AES-256, RLS PostgreSQL, soft delete
Cohérence	Forte (transactions), éventuelle (feed)	Niveau isolation	Saga pattern inter-bases

2. Justification détaillée des choix technologiques

L'architecture repose sur la Persistance Polyglotte : chaque type de données est confié à la technologie la plus adaptée à sa nature et à ses patterns d'accès. Ce choix implique une complexité opérationnelle supérieure à une solution monolithique, acceptée en contrepartie de performances optimales à grande échelle.

2.1 Tableau de synthèse

Tableau 2.1 — Mapping technologie / rôle / justification

Technologie	Rôle principal	Version	Justification principale	Alternative écartée
PostgreSQL	Profils & Auth	18.1	ACID, PostGIS, Row-Level Security RGD	MySQL (pas de RLS natif)
MongoDB	Activités sportives	7.0	Schéma JSON flexible, métriques hétérogènes	PostgreSQL JSONB (scaling limité)
Neo4j	Réseau social	5	Traversées graphe $O(k)$ vs $O(n^2)$ SQL	Table de liaison SQL (self-joins)
InfluxDB	Capteurs IoT / TS	cloud	Moteur TSM, downsampling natif, Flux	Cassandra (pas de downsampling intégré)
Kafka (MSK)	Streaming capteurs	4.1.2	Buffer durable, at-least-once, 1M msgs/s, rétention 24h	RabbitMQ (pas conçu pour le volume IoT)
Redis	Cache & Temps réel	8.4.0 cloud	Sorted Sets, Pub/Sub, < 1ms	Memcached (structures de données limitées)
Elasticsearch	Recherche full-text	9.4.0	Analyseur french, edge n-gram, geo_distance	Solr (écosystème moins mature)
Grafana	Observabilité	10	Multi-source (PG, InfluxDB, ES, Prometheus)	Kibana (ES uniquement)

2.2 Détail des justifications

PostgreSQL 18.1 — Données relationnelles et transactionnelles

Les données de profil (identité, email, mot de passe) nécessitent une cohérence forte ACID. PostgreSQL est retenu pour ses fonctionnalités avancées de sécurité et de géolocalisation :

- Row-Level Security (RLS) : chaque utilisateur ne peut accéder qu'à ses propres données sensibles — essentiel pour le RGD
- Extension pgcrypto : chiffrement AES-256 des PII (email stocké chiffré, seul le hash SHA-256 est utilisé pour la recherche)
- Extension PostGIS : requêtes géographiques ST_DWithin pour trouver des sportifs à proximité
- Partitionnement RANGE : tables comments et audit_log partitionnées par trimestre pour l'archivage automatique
- Réplication streaming native + WAL archivé vers Cloud Storage GCP pour le PITR

MongoDB 7.0 — Activités sportives

Une activité de course contient une trace GPS (tableau de coordonnées) ; une session de natation contient des splits par longueur et style. Un schéma relationnel fixe imposerait des dizaines de colonnes NULL ou une architecture EAV coûteuse.

- Document unique par activité : métriques imbriquées, splits, trace GPS dans un seul objet JSON

- Aggregation Pipeline : calcul de statistiques complexes (VO2max estimé, stats hebdo) côté serveur
- Index TTL natif sur deletedAt : purge automatique des activités supprimées après 90 jours
- Index géospatial 2dsphere sur startLocation : recherche d'activités dans un rayon donné

Neo4j 5 — Graphe social

La requête "personnes que vous connaissez peut-être" (amis d'amis) nécessite de traverser plusieurs niveaux de relations. En SQL, cela exige des self-joins multiples — $O(n^2)$. Neo4j résout cela en $O(k)$ par traversée native :

```
MATCH (me:User {id: $userId})-[:FOLLOWS]->(friend)-[:FOLLOWS]->(suggestion)
WHERE NOT (me)-[:FOLLOWS]->(suggestion) AND suggestion <> me
RETURN suggestion, COUNT(DISTINCT friend) AS mutualCount ORDER BY mutualCount DESC LIMIT 10
```

InfluxDB cloud — Capteurs IoT (séries temporelles)

- Moteur TSM (Time-Structured Merge) : compression optimisée pour les floats/timestamps — 10x plus efficace que LZ4 générique
- Flux query language : downsampling, fenêtres glissantes et détection d'anomalies de fréquence cardiaque en natif
- Tâches de rétention intégrées : données brutes 90 jours, agrégats horaires 1 an, agrégats journaliers 5 ans
- Alertes natives : seuil FC > 190 bpm déclenche un webhook → Cloud Pub/Sub → notification mobile

Redis 8.4 (cloud) — Cache et temps réel

- Sorted Sets (ZADD/ZREVRANGE) : leaderboards hebdomadaires mis à jour en $O(\log n)$ à chaque nouvelle activité
- Lists (LPUSH/LTRIM) : feed social pré-calculé, limité aux 200 dernières entrées par utilisateur
- Pub/Sub + Streams : notifications push en temps réel vers les WebSockets
- TTL natif : expiration automatique des sessions (SETEX) et du cache profils (5 min)

Redis n'est jamais source de vérité : si Redis tombe, les requêtes sont redirigées vers PostgreSQL/MongoDB avec dégradation gracieuse.

Elasticsearch 9.4 (cloud) — Recherche full-text

- Analyseur french + asciifolding : recherche de 'Vélo' trouve aussi 'velo', 'VELO'
- Edge n-gram tokenizer : autocomplétion des noms d'utilisateurs dès 2 caractères tapés
- geo_distance queries : trouver les activités dans un rayon de 10 km autour d'une position
- Index Lifecycle Management (ILM) : rotation hot/warm/cold/delete automatique

Apache Kafka (GCP MSK) — Streaming capteurs IoT

Les capteurs GPS et fréquence cardiaque génèrent ~500M points/jour avec des pics lors des heures d'entraînement. Une ingestion directe vers InfluxDB sans tampon risque de saturer la base en cas de pic. Kafka est retenu comme couche de découplage :

- Durabilité at-least-once : chaque point capteur est garanti d'être traité même en cas de redémarrage du consommateur Telegraf
- Rétention 24h sur le topic sensor-raw : permet de rejouer les données en cas d'incident InfluxDB sans perte
- Débit 1M msgs/s : 3 brokers MSK kafka.m5.large absorbent les pics d'ingestion sans backpressure
- Découplage producteur/consommateur : l'API d'ingestion n'est pas bloquée par la vitesse d'InfluxDB
- Topics séparés : sensor-raw (24h), activity-events (7j), notification-events (3j) selon la criticité

Alternative écartée : RabbitMQ n'est pas conçu pour les volumes IoT ni pour la rétention longue durée des messages.

Grafana — Observabilité unifiée

- Sources multiples dans un seul dashboard : Prometheus, InfluxDB, PostgreSQL, Elasticsearch côté à côté
- Dashboards opérationnels : santé des clusters, lag réplication PostgreSQL, mémoire Redis, débit InfluxDB
- Dashboards métier : DAU/MAU, activités par sport, croissance abonnements, volume capteurs par heure
- Alerting intégré (AlertManager) : seuils WARNING/CRITICAL avec envoi vers PagerDuty ou Slack
- Provisioning as code : dashboards versionnés en JSON dans Git, déployés automatiquement

3. Schémas conceptuels et physiques

3.1 Schéma Relationnel PostgreSQL — DDL physique complet

La base PostgreSQL héberge les données transactionnelles critiques : comptes utilisateurs, profils publics, relations de suivi, likes, commentaires et journal d'audit RGPD.

Tableau 3.1 — Table users

Colonne	Type	Contrainte	Description
id	UUID	PK DEFAULT gen_random_uuid()	Identifiant unique auto-généré
email	TEXT	NOT NULL	Chiffré AES-256 via pgp_sym_encrypt
email_hash	TEXT	NOT NULL UNIQUE	SHA-256 pour recherche sans déchiffrement
password_hash	TEXT	NOT NULL	bcrypt cost=12 (min 300ms/vérification)
role	VARCHAR(20)	CHECK IN (<i>'user'</i> , <i>'premium'</i> , <i>'coach'</i> , <i>'admin'</i>)	Rôle applicatif
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Désactivation sans suppression
deleted_at	TIMESTAMPTZ	NULL	Soft delete RGPD — NULL = compte actif
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Horodatage UTC de création

Tableau 3.2 — Table user_profiles

Colonne	Type	Contrainte	Description
user_id	UUID	PK, FK → users(id) CASCADE	Lié à users — 1:1
username	VARCHAR(50)	NOT NULL UNIQUE	Index sur lower(username) — insensible casse
display_name	VARCHAR(100)	NULL	Nom affiché publiquement
bio	TEXT	CHECK(length <= 500)	Biographie libre
city / country	VARCHAR / CHAR(2)	NULL	Localisation déclarée
geolocation	GEOGRAPHY(POINT,4326)	NULL	PostGIS — index GIST pour ST_DWithin
followers_count	INT	NOT NULL DEFAULT 0	Dénormalisé — mis à jour via trigger
is_public	BOOLEAN	NOT NULL DEFAULT TRUE	Visibilité du profil
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Trigger BEFORE UPDATE

Tableau 3.3 — Table follows

Colonne	Type	Contrainte	Description
follower_id	UUID	FK → users(id)	Utilisateur qui suit
following_id	UUID	FK → users(id)	Utilisateur suivi
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Date de l'abonnement
PK	(follower_id, following_id)	CHECK follower <> following	Auto-follow interdit

Tableau 3.4 — Table comments (partitionnée par trimestre)

Colonne	Type	Contrainte	Description
id	BIGSERIAL	NOT NULL	Auto-increment
activity_id	VARCHAR(24)	NOT NULL	ObjectId MongoDB — référence croisée
user_id	UUID	FK → users(id) ON DELETE SET NULL	Auteur (NULL si supprimé)
content	TEXT	CHECK length BETWEEN 1 AND 1000	Texte du commentaire
parent_id	BIGINT	FK → comments(id) NULL	Réponse (1 niveau de profondeur)
is_deleted	BOOLEAN	NOT NULL DEFAULT FALSE	Soft delete
created_at	TIMESTAMPTZ	NOT NULL — clé de partition	PARTITION BY RANGE(created_at)

Tableau 3.5 — Table likes

Colonne	Type	Contrainte	Description
user_id	UUID	FK → users(id)	Utilisateur qui like
activity_id	VARCHAR(24)	NOT NULL	ObjectId MongoDB
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	Horodatage
PK	(user_id, activity_id)	—	Un seul like par user par activité

Tableau 3.6 — Table notifications

Colonne	Type	Contrainte	Description
id	BIGSERIAL	PK	Auto-increment
recipient_id	UUID	FK → users(id), INDEX	Destinataire
sender_id	UUID	FK → users(id) NULL	Émetteur (NULL = système)
type	VARCHAR(30)	CHECK IN ('like', 'comment', 'follow', 'mention', 'achievement')	Type d'événement
entity_type / entity_id	VARCHAR	NOT NULL	Référence à l'entité concernée
is_read	BOOLEAN	NOT NULL DEFAULT FALSE	Index partiel

Colonne	Type	Contrainte	Description
			WHERE NOT is_read
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	TTL 30j via pg_cron

Tableau 3.7 — Table audit_log (RGPD — partitionnée par trimestre)

Colonne	Type	Contrainte	Description
id	BIGSERIAL	NOT NULL	Auto-increment
user_id	UUID	NULL	Utilisateur concerné
operator_id	UUID	NULL	DBA ou API ayant exécuté l'action
action	VARCHAR(50)	NOT NULL	'login','data_export','gdpr_delete','schema_change'
ip_address	INET	NULL	IP source — utile pour audit sécurité
details	JSONB	NULL	Métadonnées libres (ticket RGPD, bases affectées...)
created_at	TIMESTAMPTZ	NOT NULL — clé de partition	PARTITION BY RANGE(created_at)

3.2 Schéma documentaire MongoDB — Collection activities

Un document activité encapsule toutes les métriques d'une séance sportive. Le schéma est volontairement flexible : les champs varient selon le type d'activité (running, cycling, swimming...).

```
// Document exemple - activité running
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "type": "running",
  "title": "Sortie Bois de Boulogne",
  "startedAt": ISODate("2026-03-15T07:30:00Z"),
  "metrics": {
    "distance": 15.3,      // km
    "duration": 5700,      // secondes
    "avgSpeed": 9.66,      // km/h
    "avgHeartRate": 148,   // bpm
    "elevationGain": 85,   // mètres
    "calories": 1050
  },
  "startLocation": { "type": "Point", "coordinates": [2.248, 48.861] },
  "splits": [{ "km": 1, "time": 365, "avgHR": 142 }, ...],
  "visibility": "public",
  "likesCount": 12,
  "tags": ["matin", "endurance", "bois"],
  "mediaUrls": ["gs://sportconnect-media/img/act_507f.jpg"]
}
```

3.3 Schéma de graphe Neo4j

Le graphe modélise les entités sociales et leurs relations. Les nœuds User et Activity sont les entités centrales.

```
// Contraintes d'unicité
CREATE CONSTRAINT user_id_unique      FOR (u:User)      REQUIRE u.id IS UNIQUE;
CREATE CONSTRAINT activity_id_unique  FOR (a:Activity)  REQUIRE a.id IS UNIQUE;

// Nœuds
(:User      { id, username, displayName, avatarUrl })
```

```
(:Activity { id, type, visibility, createdAt })
(:Tag      { name })

// Relations
(:User)-[:FOLLOWS  { since: datetime }]->(:User)
(:User)-[:CREATED  { at: datetime }]->(:Activity)
(:User)-[:LIKED    { at: datetime }]->(:Activity)
(:Activity)-[:TAGGED_WITH]->(:Tag)
```

3.4 Schéma InfluxDB — Séries temporelles capteurs

InfluxDB stocke les données brutes des capteurs selon le Line Protocol. Trois buckets organisent la rétention par niveau d'agrégation.

```
// Line Protocol - measurement, tags fields timestamp
heart_rate,user_id=uuid123,activity_id=act456 bpm=148i,confidence=0.98 1705305000000000000
gps_speed,user_id=uuid123,activity_id=act456 speed=11.2,lat=48.861,lng=2.248
17053050010000000000

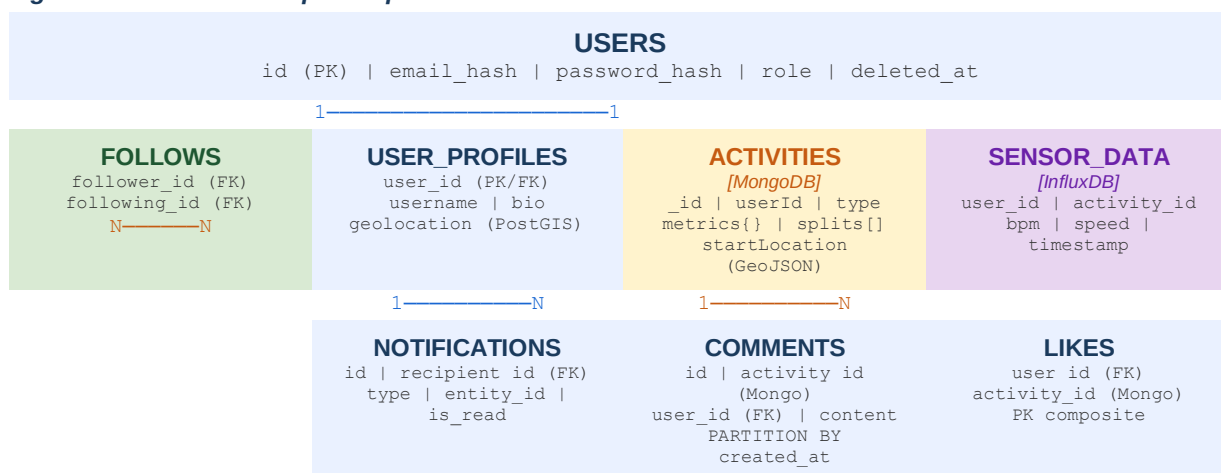
// Bucket : sensor_data | Rétention : 90 jours (données brutes)
// Bucket : sensor_1h | Rétention : 1 an (agrégat horaire - mean)
// Bucket : sensor_daily | Rétention : 5 ans (agrégat journalier - mean/max)

// Tâche Flux - downsampling horaire (exécutée toutes les heures)
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "heart_rate")
  |> aggregateWindow(every: 1h, fn: mean)
  |> to(bucket: "sensor_1h")
```

3.5 Schéma conceptuel — Diagramme Entité-Relation (ERD)

Le diagramme ci-dessous représente les entités principales de la plateforme et leurs associations. Les entités en bleu sont stockées dans PostgreSQL, en vert dans MongoDB, en orange dans Neo4j.

Figure 3.1 — ERD conceptuel SportConnect



Légende : Bleu = PostgreSQL | Jaune = MongoDB | Violet = InfluxDB | Vert = table de liaison

3.6 Architecture de stockage des médias

Les médias (photos, vidéos) représentent ~10 To/an. Ils ne transitent pas par l'API backend — les clients uploadent directement vers Cloud Storage via des URLs pré-signées, puis le CDN Cloud CDN distribue les fichiers en lecture.

Tableau 3.8 — Architecture médias Cloud Storage + Cloud CDN

Composant	Configuration	Rôle
Bucket GCS	gs://sportconnect-media europe-west1 SSE-KMS	Stockage permanent des médias
Pre-signed URL	Durée 5 min scope PUT /media/{userId}/{activityId}/*	Upload direct client → GCS sans passer par l'API
Cloud CDN	450+ Points of Presence TTL 24h pour images	Distribution mondiale en lecture
Lifecycle GCS	STANDARD → STANDARD_IA (30j) → COLDLINE (180j)	Réduction coût stockage médias anciens
Formats acceptés	JPEG, PNG, WebP (images) MP4, WebM (vidéos max 500Mo)	Validation côté API avant génération URL
Sécurité	Headers CORS stricts Content-Type vérifié antivirus scan	Protection contre uploads malveillants
URL finale	https://cdn.sportconnect.app/{userId}/{activityId}/img.jpg	Format stable pour partage et cache CDN

Flux d'upload :

- Client demande une pre-signed URL à l'API (authentié)
- L'API vérifie les droits, génère l'URL signée GCS (TTL 5 min) et la retourne
- Le client uploade le fichier directement vers GCS en PUT (sans passer par l'API)
- GCS déclenche un event Pub/Sub → traitement asynchrone (resize, thumbnail, scan antivirus)
- L'URL finale est stockée dans le document MongoDB (mediaUrls[]) de l'activité
- Cloud CDN met en cache l'image sur les PoPs au premier accès (TTL 24h)

3.7 Flux temps réel des capteurs IoT

Les capteurs GPS et de fréquence cardiaque génèrent ~500M points/jour. Le flux passe par Kafka (MSK) pour absorber les pics, puis Telegraf ingère dans InfluxDB Cloud.

Figure 3.2 — Pipeline de données capteurs bout en bout

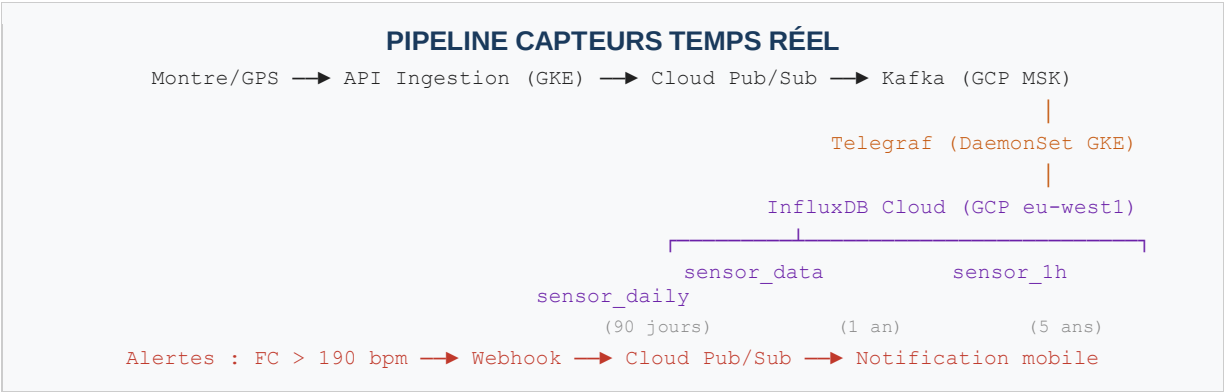


Tableau 3.9 — Composants du pipeline capteurs

Composant	Technologie	Rôle	Débit cible
Appareils clients	SDK mobile (iOS/Android)	Collecte GPS (1Hz) et FC (1Hz), bufferisation locale si offline	—

Composant	Technologie	Rôle	Débit cible
API Ingestion	Microservice GKE (Python/FastAPI)	Validation, authentification, push vers Pub/Sub	50 000 req/s
Cloud Pub/Sub	GCP Pub/Sub — topic sensor-raw	Buffer durable entre ingestion et traitement, garantie at-least-once	500M msgs/j
Kafka (GCP MSK)	Apache Kafka 3 brokers	File d'attente résistante aux pics — rétention 24h sur sensor-raw	1M msgs/s
Telegraf	DaemonSet GKE — 1 pod / nœud	Consomme Kafka, batch de 5000 pts, ingère vers InfluxDB Cloud	500K pts/s
InfluxDB Cloud	Serverless GCP eu-west1	Stockage TSM, downsampling Flux automatique, alertes FC	Illimité (serverless)
Fallback Redis	Streams Redis — key sensor:buffer	Absorbe le surplus si InfluxDB ralentit (backpressure)	Tampon < 10 min

3.8 Cohérence des données inter-bases — Pattern Saga

Une opération comme 'publier une activité' touche simultanément MongoDB, Neo4j, Elasticsearch, PostgreSQL et Redis. En l'absence de transaction distribuée (trop coûteuse en latence), le pattern Saga garantit la cohérence éventuelle : chaque étape est soit exécutée, soit compensée (annulée).

Tableau 3.10 — Saga : publication d'une activité (CreateActivitySaga)

Étape	Base	Action	Compensation si échec ultérieur
1	MongoDB	INSERT activité (source de vérité)	DELETE activité MongoDB
2	Neo4j	CREATE (User)-[:CREATED]->(Activity)	DETACH DELETE nœud Activity
3	Elasticsearch	INDEX document activité	DELETE document ES
4	PostgreSQL	INCREMENT activities_count (user_profiles)	DECREMENT activities_count
5	Redis	INVALIDATE feed des followers	Aucune (cache déjà invalidé)

Principe : si l'étape 3 (Elasticsearch) échoue, les compensations 2 et 1 sont exécutées en ordre inverse. L'activité n'est jamais visible partiellement. Le délai de cohérence acceptable est fixé à 5 secondes maximum.

Tableau 3.11 — Saga : droit à l'oubli RGPD (DeleteUserSaga)

Étape	Base	Action	Compensation
1	PostgreSQL	Anonymisation email + soft-delete (is_active=FALSE)	Alerte manuelle — RGPD irréversible
2	MongoDB	DELETE activités, notifications	Alerte manuelle
3	Neo4j	DETACH DELETE nœud User	Alerte manuelle
4	Redis	Purge clés cache et sessions	Aucune (cache)
5	Elasticsearch	delete_by_query sur activités et users	Alerte manuelle
6	InfluxDB	DELETE mesures capteurs WHERE user_id	Alerte manuelle
7	PostgreSQL	INSERT dans audit_log (ticket RGPD)	Aucune (log)

Note : pour le droit à l'oubli, les compensations ne restaurent pas les données — elles déclenchent une alerte PagerDuty pour intervention manuelle et traçabilité RGPD.

4. Procédures d'administration complètes

4.1 Gestion des rôles — Principe du moindre privilège

Chaque composant applicatif dispose d'un rôle dédié avec les permissions minimales nécessaires. Aucun service n'utilise le compte administrateur en production.

Tableau 4.1 — Rôles PostgreSQL

Rôle	Accordé à	Permissions	Restriction
sc_readonly	Reporting, analytics	SELECT sur toutes les tables	Aucun INSERT/UPDATE/DELETE
sc_app	API backend (GKE pods)	SELECT,INSERT,UPDATE sur tables métier + INSERT audit_log	Pas de DROP, ALTER, DELETE massif
sc_dba	DBA uniquement	ALL PRIVILEGES	Accès via Cloud SQL Auth Proxy uniquement

Tableau 4.2 — Rôles MongoDB (Atlas)

Rôle	Accordé à	Permissions
sc_app_rw	API backend	readWrite sur db sportconnect uniquement
sc_admin	DBA	dbAdmin + clusterMonitor — pas d'accès aux données utilisateur
sc_analytics	Grafana, reporting	read uniquement — pour les agrégations Grafana

4.2 Monitoring et alertes

Stack de monitoring : Cloud Monitoring GCP (services natifs) + Grafana (agrégation multi-sources) + AlertManager (routage des alertes).

Tableau 4.3 — Seuils d'alerte par service

Service	Métrique surveillée	Seuil WARNING	Seuil CRITICAL	Canal d'alerte
PostgreSQL	Lag de réplication	10 MB	100 MB	PagerDuty
PostgreSQL	Connexions actives	150 / 200	180 / 200	Slack #ops
MongoDB	Opérations en attente	100	500	PagerDuty
Redis	Cache hit rate	< 85%	< 70%	Slack #ops
InfluxDB	Latence d'écriture	50ms	200ms	Slack #ops
Kafka	Consumer lag (sensor-raw)	10 000 msgs	100 000 msgs	PagerDuty
Kafka	Disk usage brokers	70%	85%	Slack #ops
Elasticsearch	Heap JVM	75%	90%	PagerDuty
Tous services	CPU / RAM	80% / 5min	95% / 2min	PagerDuty

4.3 Administration Kafka — Gestion des topics et consumers

Kafka requiert une administration spécifique : gestion des topics, surveillance du consumer lag, et rotation des offsets en cas de rejeu de données.

Tableau 4.4 — Topics Kafka SportConnect

Topic	Partitions	Réplication	Rétention	Consommateurs
sensor-raw	12	3	24h	Telegraf (InfluxDB), Redis Streams (fallback)
activity-events	6	3	7j	Neo4j Sync, Elasticsearch Indexer, Notification Service
notification-events	3	3	3j	Push Notification Service, WebSocket Gateway
gdpr-delete-events	1	3	30j	Saga Orchestrator — suppression coordonnée

```
# Créer un topic Kafka (via CLI MSK)
kafka-topics.sh --bootstrap-server $KAFKA_BROKERS \
  --create --topic sensor-raw \
  --partitions 12 --replication-factor 3 \
  --config retention.ms=86400000

# Surveiller le consumer lag
kafka-consumer-groups.sh --bootstrap-server $KAFKA_BROKERS \
  --describe --group telegraf-sensor

# Rejouer depuis un offset précis (incident InfluxDB)
kafka-consumer-groups.sh --bootstrap-server $KAFKA_BROKERS \
  --group telegraf-sensor --reset-offsets \
  --topic sensor-raw --to-datetime 2026-03-15T06:00:00.000 --execute
```

4.5 Conformité RGPD — Données de santé

SportConnect collecte des données sensibles de santé (fréquence cardiaque, géolocalisation précise). Les mesures suivantes sont impératives :

Chiffrement des données personnelles

```
-- Email chiffré à l'insertion (jamais stocké en clair)
INSERT INTO users (email, email_hash, password_hash) VALUES (
  pgp_sym_encrypt('user@example.com', current_setting('app.encryption_key')),
  encode(sha256('user@example.com'::bytea), 'hex'),
  crypt('password', gen_salt('bf', 12)) -- bcrypt cost=12
);
```

Procédure droit à l'oubli (suppression en cascade sur toutes les bases)

- PostgreSQL : anonymisation email → hash aléatoire, is_active=FALSE, deleted_at=NOW()
- MongoDB : suppression de toutes les activités, commentaires, likes et notifications
- Neo4j : DETACH DELETE du nœud User (supprime toutes ses relations)
- InfluxDB : DELETE FROM heart_rate WHERE user_id = \$userId
- Redis : suppression de toutes les clés contenant l'user_id (cache, feed, sessions)
- Elasticsearch : delete_by_query sur index activities et users
- Toute suppression est journalisée dans audit_log avec le ticket RGPD associé

4.6 Initialisation de l'infrastructure

```
# Démarrage de la stack locale (développement)
docker compose up -d

# Initialisation PostgreSQL
psql -h localhost -U sportal_admin -d sportconnect -f scripts/sql/01_schema.sql
```



```
# Initialisation MongoDB
python scripts/mongodb/mongodb_setup.py

# Setup GCP (production)
python scripts/gcp/gcp_setup.py

# Health check tous services
python scripts/admin/admin.py health-check
```

5. Plan de sauvegarde et reprise d'activité (PRA)

5.1 Objectifs de récupération

Tableau 5.1 — RPO, RTO et MTTR

Métrique	Définition	Objectif SportConnect	Moyen technique
RPO	Perte de données max acceptable	< 1 heure (données critiques)	WAL continu PG + oplog MongoDB → GCS
RTO	Durée max d'indisponibilité	< 4 heures (restauration complète)	Réplicas + failover automatique Cloud SQL
MTTR	Temps moyen de rétablissement	< 2 heures	Runbooks automatisés + monitoring actif

5.2 Stratégie de sauvegarde par technologie

Tableau 5.2 — Plan de sauvegarde

Base	Type	Fréquence	Rétention	Outil / Destination GCS
PostgreSQL	Physique (base complète)	Quotidien 02h00	30 jours	pg_basebackup --gzip → gs://sportconnect-backups/postgresql/
PostgreSQL	WAL incrémental (PITR)	Continue	7 jours	archive_command → gs://sportconnect-backups/wal/
MongoDB	Logique + oplog	Quotidien 03h00	14 jours	mongodump --oplog --gzip → gs://sportconnect-backups/mongodb/
InfluxDB	Snapshot cloud	Géré par InfluxDB Cloud	30 jours	UI InfluxDB Cloud (GCP eu-west1)
Redis	RDB snapshot (BGSAVE)	Toutes les 6h	7 jours	dump.rdb → gs://sportconnect-backups/redis/
Neo4j	Snapshot AuraDB	Géré par AuraDB	7 jours	Console AuraDB (automatique)
Kafka (MSK)	Snapshot MSK	Géré par AWS MSK	7 jours	Console MSK — snapshots automatiques
Elasticsearch	Snapshot GCS	Quotidien 05h00	30 jours	PUT /_snapshot/gcs_repo/<snapshot>

5.3 Scénarios de panne et procédures

Scénario 1 : Perte d'un nœud PostgreSQL secondaire

- Le nœud primaire continue de servir les requêtes sans interruption de service
- Alerting automatique (Cloud Monitoring → PagerDuty) en < 1 minute
- Provisionnement d'un nouveau réplica Cloud SQL depuis le dernier backup physique
- Replay des WAL archivés depuis GCS pour rattraper le retard
- Failover automatique géré par Cloud SQL si le primaire est affecté

Scénario 2 : Corruption de données MongoDB

- Arrêt des écritures sur la collection affectée (circuit breaker applicatif)

- Identification du dernier état cohérent via l'oplog MongoDB
- Restauration point-in-time : mongorestore + oplog replay jusqu'au moment avant corruption
- Validation des données restaurées via checksum avant réouverture du service

Scénario 3 : Saturation InfluxDB (pic d'ingestion)

- Redis Streams absorbe le surplus en tampon temporaire
- Activation automatique du mode batch : regrouper les points d'une même seconde
- Scale-out applicatif : ajout d'instances d'API derrière le Load Balancer GCP

Scénario 4 : Panne d'un broker Kafka

- Les 2 brokers restants continuent de servir les partitions (réplication factor = 3)
- Alerting Cloud Monitoring sur la métrique UnderReplicatedPartitions en < 2 minutes
- MSK provisionne automatiquement un broker de remplacement dans la même zone
- Resynchronisation automatique des partitions sous-répliquées sans interruption de service
- Si tous les brokers tombent : les messages bufférisés dans Pub/Sub (24h) permettent le rejeu complet

Scénario 5 : Panne du nœud Redis primaire

- Memorystore Standard déclenche un failover automatique vers le réplica en < 30 secondes
- Pendant le failover : les requêtes de lecture sont redirigées vers PostgreSQL/MongoDB (dégradation gracieuse)
- Cache hit rate chute temporairement à 0% — alerting Slack immédiat
- Redis ne stockant jamais de source de vérité, aucune perte de données applicative
- Reconstruction du cache : les clés se re-peuplent automatiquement à la première requête (lazy loading)

5.4 Tests de PRA obligatoires

Un PRA non testé n'existe pas. Tout scénario de reprise doit être validé périodiquement.

Tableau 5.3 — Calendrier des tests PRA

Fréquence	Test	Environnement	Critère de succès
Mensuel	Restauration PostgreSQL depuis pg_basebackup	Staging isolé	RTO < 1h, données cohérentes
Mensuel	Restauration MongoDB avec oplog replay	Staging isolé	Zéro perte après point-in-time
Trimestriel	Failover Cloud SQL (bascule primaire → réplica)	Pré-production	Basculement < 60s
Trimestriel	Restauration snapshot InfluxDB Cloud	Staging isolé	Données brutes et agrégats OK
Annuel	Simulation perte complète d'une zone GCP	Simulation DR	RTO < 4h, RPO < 1h

6. Stratégie de mise à l'échelle

6.1 Mécanismes de scaling par technologie

Tableau 6.1 — Stratégies de scaling

Technologie	Mécanisme	Configuration initiale	Limite / Seuil de migration
PostgreSQL (Cloud SQL)	Read replicas	1 primaire + 1 réplica	Sharding via AlloyDB si > 10M utilisateurs
MongoDB (Atlas)	Sharding natif	Shard key: hashed userId, 3 shards	Config servers répliqués (3 nœuds)
Kafka (MSK)	Ajout de brokers + partitions	3 brokers, 12 partitions sensor-raw	Scale horizontal — ajout broker sans downtime
InfluxDB Cloud	Serverless auto-scale	Usage-based — scale automatique	Migration vers VictoriaMetrics si coût > seuil
Redis (Memorystore)	Réplica Standard Tier	1 primaire + 1 réplica	Redis Cluster si > 25 Go données actives
Neo4j (AuraDB)	Managed clustering	1 writer + replicas	Scale up instance AuraDB
Elasticsearch (Elastic Cloud)	Sharding + replicas	3 data nodes hot-warm	Hot-warm-cold pour vieux index

6.2 Projections de croissance

Tableau 6.2 — Actions infrastructure par horizon

Horizon	Utilisateurs	Activités/jour	Actions infrastructure
Lancement	10 000	50 000	Cloud SQL single-AZ, MongoDB Atlas M10, Redis 1Go — coût ~300€/mois
6 mois	100 000	500 000	Cloud SQL Multi-AZ, Atlas M30 + read replicas, Redis 5Go
18 mois	1 000 000	5 000 000	Atlas sharding, Elastic Cloud multi-nodes, Memorystore Standard
36 mois	10 000 000	50 000 000	Multi-région GCP, AlloyDB, CDN médias global, GKE Autopilot

6.3 Stratégie de cache Redis

Redis absorbe une grande partie des lectures répétitives, réduisant la pression sur les bases principales.

Tableau 6.3 — Politique de cache

Clé Redis	Structure	TTL	Invalidation
profile:{username}	STRING (JSON)	300s (5 min)	Immédiate sur mise à jour profil
feed:{user_id}	LIST (200 max)	600s (10 min)	Suppression sur nouveau follow ou nouvelle activité
leaderboard:{sport}:{week}	SORTED SET	1 209 600s (14j)	Incrémentiel : ZINCRBY à chaque activité
likes:{activity_id}	SET	86 400s (24h)	SADD/SREM à chaque like/unlike
session:{token}	STRING (JSON)	3 600s (1h)	SETEX — expiration automatique

Cible de hit rate Redis : > 85%. En dessous, révision de la politique de cache nécessaire.

7. Connexion des technologies à Google Cloud Platform

Cette section documente comment chaque technologie retenue est connectée et déployée sur GCP. Le principe directeur est de privilégier les services managés GCP ou les offres cloud des éditeurs hébergées sur GCP, afin de réduire la charge opérationnelle tout en restant dans l'écosystème Google Cloud.

7.1 Vue d'ensemble — Mapping technologies / services GCP

Tableau 7.1 — Correspondance technologie choisie / service GCP

Technologie	Service GCP / Partenaire	Région	Mode de connexion	Compatibilité
PostgreSQL	Cloud SQL for PostgreSQL 16	europe-west1	VPC Private IP + Cloud SQL Auth Proxy	Totale
MongoDB	MongoDB Atlas on GCP	europe-west1	VPC Peering Atlas ↔ GCP	Totale (API identique)
Neo4j	Neo4j AuraDB (hébergé GCP)	europe-west4	HTTPS Bolt + IP Allowlist CIDR GKE	Totale (Cypher)
InfluxDB	InfluxDB Cloud Serverless (GCP)	eu-west1	HTTPS via NAT Gateway IP fixe	Totale (HTTP API)
Kafka	GCP Pub/Sub + MSK (kafka.m5.large)	europe-west1	VPC interne GKE — SASL/SCRAM	Totale
Redis	Memorystore for Redis	europe-west1	VPC Private IP (natif GCP)	Totale
Elasticsearch	Elastic Cloud on GCP	europe-west1	HTTPS + Traffic Filter CIDR GKE	Totale (API identique)
Médias	Cloud Storage + Cloud CDN	Global	Pre-signed URLs (upload direct)	Native GCP

7.2 Architecture réseau VPC

Tableau 7.2 — Configuration réseau GCP

Composant	Configuration	Rôle
VPC principal	sc-vpc (10.0.0.0/16) — europe-west1	Réseau privé commun à tous les services
Subnet GKE	sc-gke-subnet (10.0.1.0/24) — alias IPs activées	Pods microservices SportConnect
Subnet services	sc-services-subnet (10.0.2.0/24) — Private Service Access	Cloud SQL, Memorystore
Cloud NAT	NAT Gateway europe-west1 — IP fixe sortante	Connexion vers Atlas, AuraDB, Elastic, InfluxDB Cloud
Firewall rules	Allow GKE → SQL:5432, Redis:6379 / Deny all par défaut	Isolation réseau stricte par service
Private Google Access	Activé sur tous les subnets	Accès APIs Google (KMS, Secret Manager, GCS) sans internet

7.3 Configuration détaillée par service

Cloud SQL for PostgreSQL

- Instance : db-custom-4-16384 (4 vCPU / 16 Go RAM), SSD 500 Go, IOPS 6000
- High Availability : Multi-AZ activé, failover automatique < 60s
- Connexion : Cloud SQL Auth Proxy (sidecar GKE) — mTLS automatique, pas de mot de passe en clair
- Backups : quotidiens 02h00 UTC, rétention 35 jours, PITR activé (WAL → GCS)
- Chiffrement : CMEK via Cloud KMS, TLS 1.3 obligatoire

MongoDB Atlas on GCP

- Cluster M30 : 3 nœuds (8 vCPU / 32 Go / 640 Go SSD chacun), region GCP europe-west1
- VPC Peering : communication par IP privée sans transit internet entre Atlas et GKE
- Backups : Cloud Backup Atlas, PITR via oplog, rétention 14 jours
- Chiffrement : CMEK via Google Cloud KMS (Atlas Encryption at Rest)

Neo4j AuraDB on GCP

- Instance Professional : 8 Go RAM, SSD managé, réplication automatique multi-zone
- Connexion : Bolt protocol (neo4j+s://) TLS 1.3 — IP Allowlist restreinte au CIDR NAT GKE
- Backups : snapshots automatiques quotidiens gérés par Neo4j (rétention 7 jours)

InfluxDB Cloud Serverless (GCP eu-west1)

- Buckets : sensor_data (90j), sensor_1h (1an), sensor_daily (5ans)
- Ingestion : Telegraf DaemonSet GKE → HTTPS InfluxDB Cloud
- Authentification : API Token (scope write uniquement) dans Secret Manager
- Alerting natif : FC > 190 bpm → webhook → Cloud Pub/Sub → notification mobile

Memorystore for Redis (GCP natif)

- Standard Tier 5 Go : 1 primaire + 1 réplica, zones europe-west1-b / europe-west1-c
- Connexion : IP privée uniquement (10.0.2.x:6379) — pas d'accès externe possible
- Failover automatique < 30s sur le réplica en cas de panne du nœud primaire
- Eviction Policy : allkeys-lru — libération automatique si mémoire pleine

Apache Kafka — GCP Pub/Sub + MSK

- Cluster MSK : 3 brokers kafka.m5.large (2 vCPU / 8 Go RAM), réplication factor = 3
- Topics : sensor-raw (12 partitions, 24h), activity-events (6 partitions, 7j), notification-events (3 partitions, 3j)
- Connexion depuis GKE : SASL/SCRAM via Private Link dans le VPC — pas d'accès externe
- Authentification : credentials SASL dans Secret Manager, rotation trimestrielle
- Schema Registry : AWS Glue Schema Registry pour validation Avro des messages capteurs
- Monitoring : métriques MSK → CloudWatch → Grafana (consumer lag, throughput, broker health)

Elastic Cloud on GCP

- Déploiement : 2 data nodes r6g.large (4 Go RAM chacun) + 1 master, hot-warm
- Traffic Filter : restreint les connexions au CIDR de la NAT Gateway GKE uniquement
- Authentification : API Key (scope index write + search) — Basic Auth désactivé
- ILM : hot (0-30j) → warm (30-180j) → cold (180j-1an) → delete

7.4 Gestion des accès — IAM et Secret Manager

Tous les credentials sont centralisés dans GCP Secret Manager. Les pods GKE accèdent aux secrets via Workload Identity (sans fichier de credentials sur disque).

Tableau 7.3 — Matrice IAM par service

Service	Méthode d'authentification	Rôle IAM	Remarque
Cloud SQL	Workload Identity (GKE SA → IAM SA)	roles/cloudsql.client	Auth Proxy gère mTLS automatiquement
MongoDB Atlas	Secret Manager (SCRAM credentials)	secretmanager.secretAccessor	Rotation manuelle trimestrielle
Neo4j AuraDB	Secret Manager (Bolt credentials)	secretmanager.secretAccessor	IP Allowlist CIDR NAT GKE
InfluxDB Cloud	Secret Manager (API Token)	secretmanager.secretAccessor	Token scope write uniquement
Kafka (MSK)	Secret Manager (SASL/SCRAM user+pass)	secretmanager.secretAccessor	Private Link VPC — pas d'accès public
Memorystore Redis	Secret Manager (AUTH string)	redis.viewer (monitoring only)	Private IP — pas d'IAM direct
Elastic Cloud	Secret Manager (API Key)	secretmanager.secretAccessor	Traffic Filter sur CIDR GKE NAT
Cloud Storage	Workload Identity (GKE SA → IAM SA)	roles/storage.objectCreator	Bucket médias uniquement

Annexe A — Architecture Decision Records (ADR)

Les ADR documentent les décisions architecturales importantes, leurs justifications et leurs compromis, pour la traçabilité et l'onboarding des nouveaux membres d'équipe.

ADR-001 : InfluxDB Cloud plutôt que Cassandra pour les capteurs

Décision : InfluxDB Cloud Serverless (zone GCP)

Contexte : Ingestion de ~500M points/jour de capteurs GPS et fréquence cardiaque

Raison : Moteur TSM natif, downsampling intégré, clustering inclus en version Cloud

Compromis : Coût variable selon volume — monitoring budget requis

Conséquence : Alertes coût configurées dans Cloud Billing dès 80% du budget mensuel

ADR-002 : Rejet d'une architecture mono-base (PostgreSQL + TimescaleDB)

Option rejetée : Tout dans PostgreSQL avec JSONB + TimescaleDB + pgvector

Raison du rejet : Viable pour un MVP < 100K utilisateurs, souffre à grande échelle

Problèmes : Contention WAL partagé, scaling vertical uniquement TimescaleDB OSS, graphe social inefficace

Décision : Architecture polyglotte acceptée malgré la complexité opérationnelle supérieure

ADR-003 : Cohérence éventuelle pour le feed social

Décision : Cohérence éventuelle acceptable pour le fil d'actualité

Justification : Consistance forte nécessiterait des transactions distribuées (2PC) trop coûteuses en latence

Conséquence : Pattern Saga pour la cohérence inter-bases — délai max accepté : 5 secondes

ADR-004 : Neo4j AuraDB on GCP plutôt que Amazon Neptune

Décision : Neo4j AuraDB hébergé sur GCP (europe-west4)

Raison : Cypher natif, compatibilité directe avec le code existant, même écosystème GCP

Compromis : Endpoint public (sécurisé TLS + IP Allowlist) vs Neptune en VPC pur

ADR-005 : Apache Kafka comme couche de streaming entre capteurs et InfluxDB

Décision : GCP Pub/Sub + MSK (Apache Kafka 3 brokers kafka.m5.large)

Contexte : 500M points/jour avec des pics aux heures d'entraînement — ingestion directe vers InfluxDB risquée

Raison : Découplage producteur/consommateur, rétention 24h pour replay, débit 1M msgs/s garanti

Alternative écartée : RabbitMQ — pas conçu pour le volume IoT ni pour la rétention longue durée

Conséquence : Délai d'ingestion max de 5 secondes entre la mesure et le stockage dans InfluxDB

Annexe B — Glossaire technique

Tableau B.1 — Termes et définitions

Terme	Définition
ACID	Atomicité, Cohérence, Isolation, Durabilité — propriétés des transactions relationnelles
ADR	Architecture Decision Record — document traçant une décision architecturale et ses justifications
AOF	Append-Only File — journal de toutes les opérations Redis pour la durabilité
CMEK	Customer Managed Encryption Key — clé KMS gérée par le client pour le chiffrement GCP
Flux	Langage de requête fonctionnel natif d'InfluxDB, optimisé pour les séries temporelles
IaC	Infrastructure as Code — infrastructure définie et versionnée en code (Terraform)
ILM	Index Lifecycle Management — gestion automatique du cycle de vie des index Elasticsearch
Kafka	Plateforme de streaming distribuée — découplage producteur/consommateur, rétention configurable
PITR	Point-In-Time Recovery — capacité à restaurer une base à un instant précis
PII	Personally Identifiable Information — données à caractère personnel (email, géoloc...)
PostGIS	Extension PostgreSQL ajoutant le support des données géospatiales (géométries, distances)
RPO	Recovery Point Objective — perte de données maximale acceptable en cas de sinistre
RLS	Row-Level Security — mécanisme PostgreSQL d'isolation des données par utilisateur
RTO	Recovery Time Objective — durée maximale d'indisponibilité acceptable
Saga	Pattern de gestion des transactions distribuées sans verrou global (2PC)
Sharding	Partitionnement horizontal des données sur plusieurs nœuds physiques
TSM	Time-Structured Merge — moteur de stockage InfluxDB optimisé pour les timestamps
TTL	Time To Live — durée de vie d'une donnée avant suppression automatique
VPC	Virtual Private Cloud — réseau privé isolé dans GCP
WAL	Write-Ahead Log — journal de transactions PostgreSQL pour la durabilité et la réplication
Workload Identity	Mécanisme GCP liant un Service Account Kubernetes à un IAM SA (sans fichier de clé)

— Fin du document —

[SportConnect](#) | [Dossier d'Architecture Technique](#) | v2.0